FPT University

HO CHI MINH CITY CAMPUS

.....



Linux and Open Source Platform

Assignment 2:

Writing Embedded Application & Sensor Monitoring System

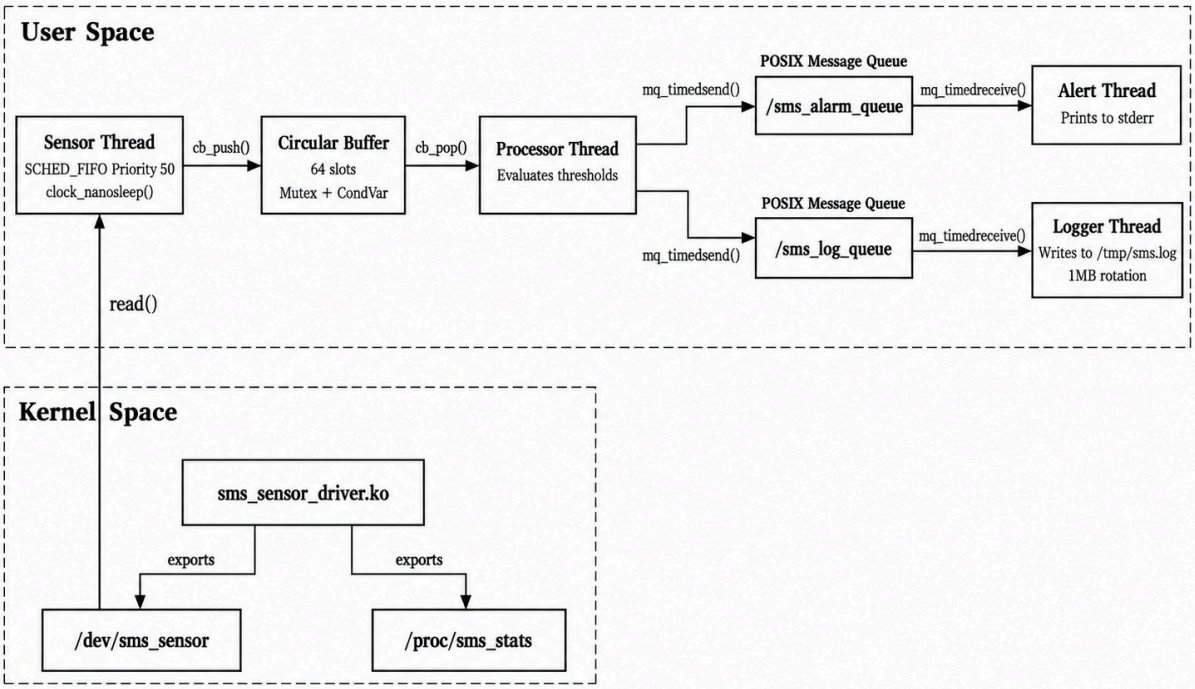
Name	Roll number	Subject
Hoàng Ngọc Gia Bão	SE203238	LOS201
Trần Hồ Khánh Tân	SE203740	LOS201
Nguyễn Trọng Hùng	SE204352	LOS201
Nguyễn Quế Vũ	SE204290	LOS201
Nguyễn Việt Anh	SE204364	LOS201
Hồ Phan Anh Tuấn	SE203437	LOS201

Lecturer: Phạm Thế Vinh

Ho Chi Minh City, Oct 17, 2026

1. SYSTEM ARCHITECTURE & DESIGN DECISIONS (TASK 1 & TASK 5.1)

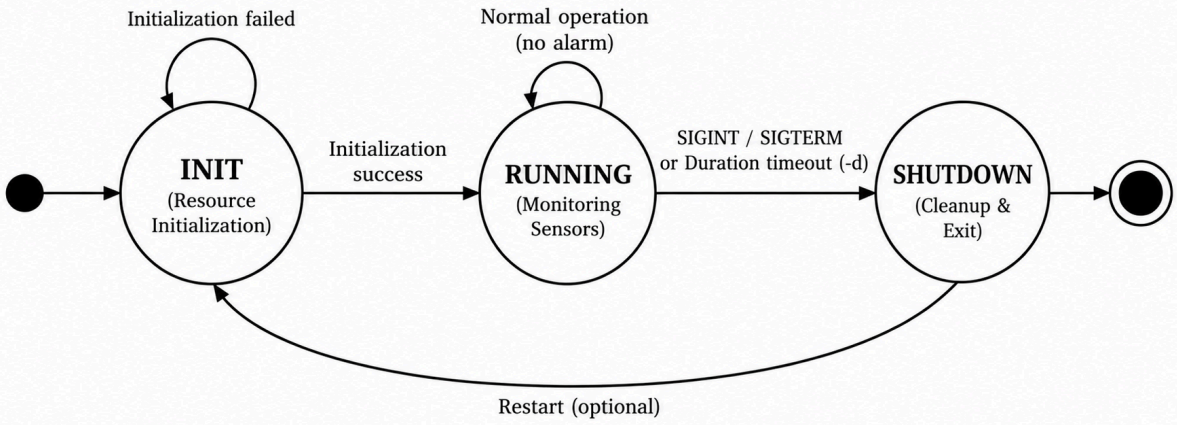
The Sensor Monitoring System (SMS) is an industrial-grade embedded Linux application designed to continuously sample environmental parameters from a simulated kernel hardware driver, buffer high-rate samples through a deterministic circular ring buffer, perform anomaly detection, and distribute alerts/logs via asynchronous POSIX IPC pipelines.



[FIGURE 1] architecture_diagram.png — *System Architecture & End-to-End Inter-Process Communication (IPC) Data Flow Diagram*

1.1. State Machine Specifications

The userspace daemon transitions across three well-defined execution states, guaranteeing predictable startup, deterministic runtime sampling, and clean signal termination:



[FIGURE 2] state_machine_diagram.png — *Finite State Machine (FSM) Lifecycle: Initialization, Real-Time Execution, and Graceful Shutdown*

1.2. Justification of Critical Architectural Decisions

- **1. Circular Ring Buffer vs. Linked List:** Pre-allocated on the heap once during initialization with a capacity of 64 slots. Unlike linked lists, a circular ring buffer completely eliminates dynamic malloc()/free() calls during real-time operation, preventing heap fragmentation, cache pollution, and unpredictable OS page fault latency spikes.
- **2. POSIX Message Queues vs. Shared Memory:** POSIX MQs preserve discrete message boundaries, provide kernel-enforced prioritization (alerts preempt normal logs), and offer native asynchronous blocking semantics (mq_timedreceive). This avoids the severe overhead, custom spinlocks, and complex race conditions associated with raw shared memory.
- **3. Dedicated Logger Thread vs. Direct File I/O:** Decouples high-priority real-time data acquisition from high-latency disk I/O operations (file writes, periodic flushing, 1MB log rotation). By delegating I/O to an independent worker thread, the Sensor and Processor threads remain strictly non-blocking.

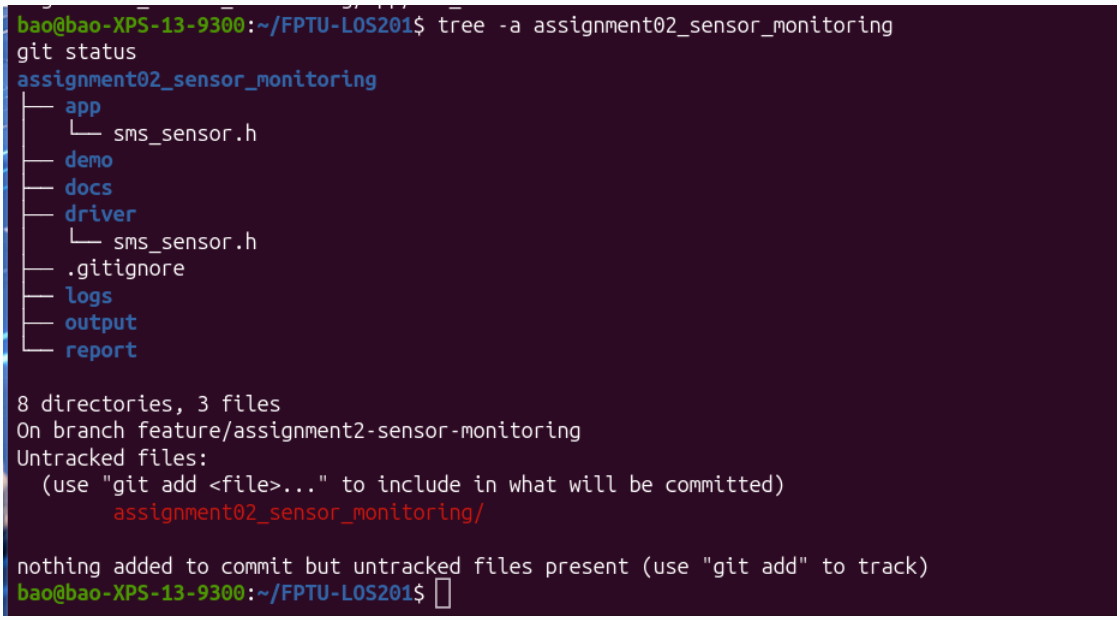
1.3. Multi-Threading Pipeline Architecture

Thread Name	Scheduling Policy	Priority	Periodicity / Trigger	Communication / Output
Main Coordinator	SCHED_OTHER	0 (Default)	Signals (SIGINT / SIGTERM)	pthread_join(), summary reporting, heap cleanup
Sensor Thread	SCHED_FIFO / OTHER	RT 50 / 0	Periodic 500ms (clock_nanosleep)	Reads /dev/sms_sensor, cb_push() into buffer

Processor Thread	SCHED_OTHER	0 (Default)	Event-driven (cb_pop condvar)	10-sample rolling avg, mq_timedsend() to queues
Logger Thread	SCHED_OTHER	0 (Default)	Event-driven (mq_timedreceive)	Writes /tmp/sms.log, 1MB rotation, 5s flush
Alert Thread	SCHED_OTHER	0 (Default)	Event-driven (mq_timedreceive)	Real-time critical warning output to stderr

2. CORE MODULE IMPLEMENTATION & IPC VERIFICATION (TASK 1)

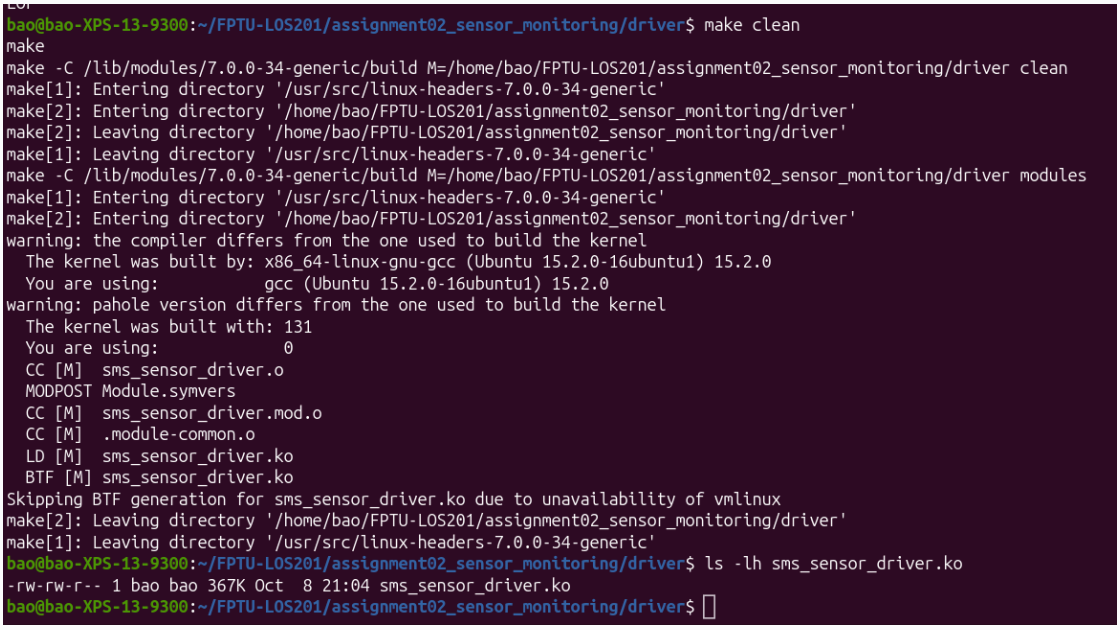
2.1. Project Directory Structure: The workspace separates kernel module drivers, userspace multi-threaded daemon code, verifiable logs, and experimental documentation:



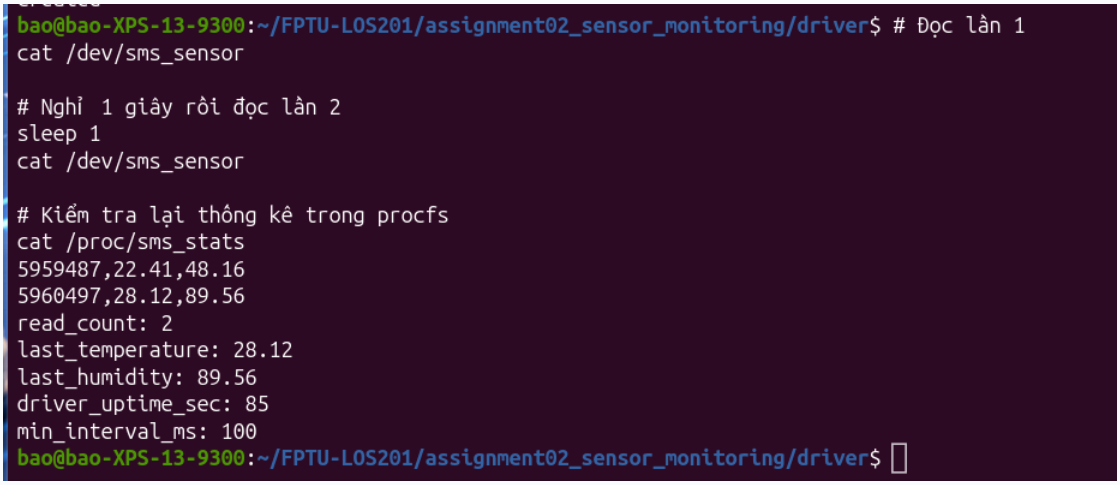
[EVIDENCE 01] 01_project_structure.png — *Standardized assignment directory hierarchy containing isolated driver, app, logs, and docs modules*

2.2. Kernel Character Driver (sms_sensor_driver.ko): Implemented as an out-of-tree Loadable Kernel Module (LKM) registering a character device at /dev/sms_sensor with dynamic major allocation. The driver implements read(), open(), release(), and llseek() file operations, returning CSV records formatted as '<timestamp_ms>,<temperature>,<humidity>\n'. The driver enforces rate-limiting (g_min_interval_ms = 100ms): if userspace reads faster than the configured threshold, it returns -EAGAIN, preventing bus

congestion. Runtime interval adjustments are supported via ioctl(SMS_SET_INTERVAL), and telemetry counters are exported via /proc/sms_stats.



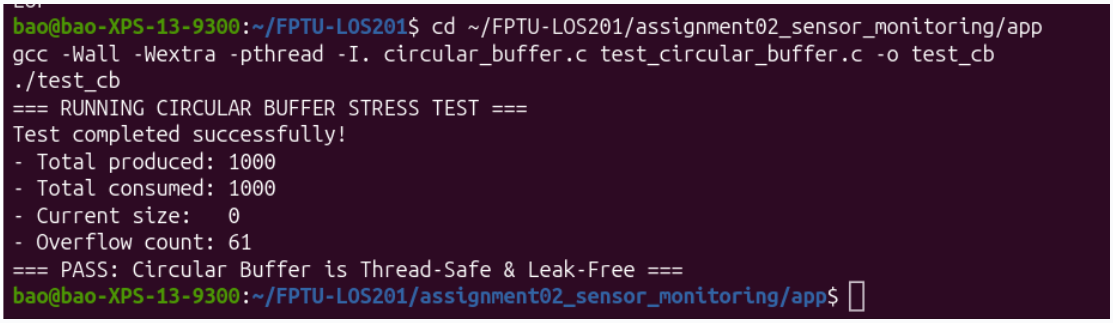
[EVIDENCE 02] 02_driver_build.png — *Clean compilation and build verification of the Linux kernel character driver module*



[EVIDENCE 03] 03_driver_test.png — *Driver loading via insmod, device node validation under /dev/sms_sensor, and telemetry output from /proc/sms_stats*

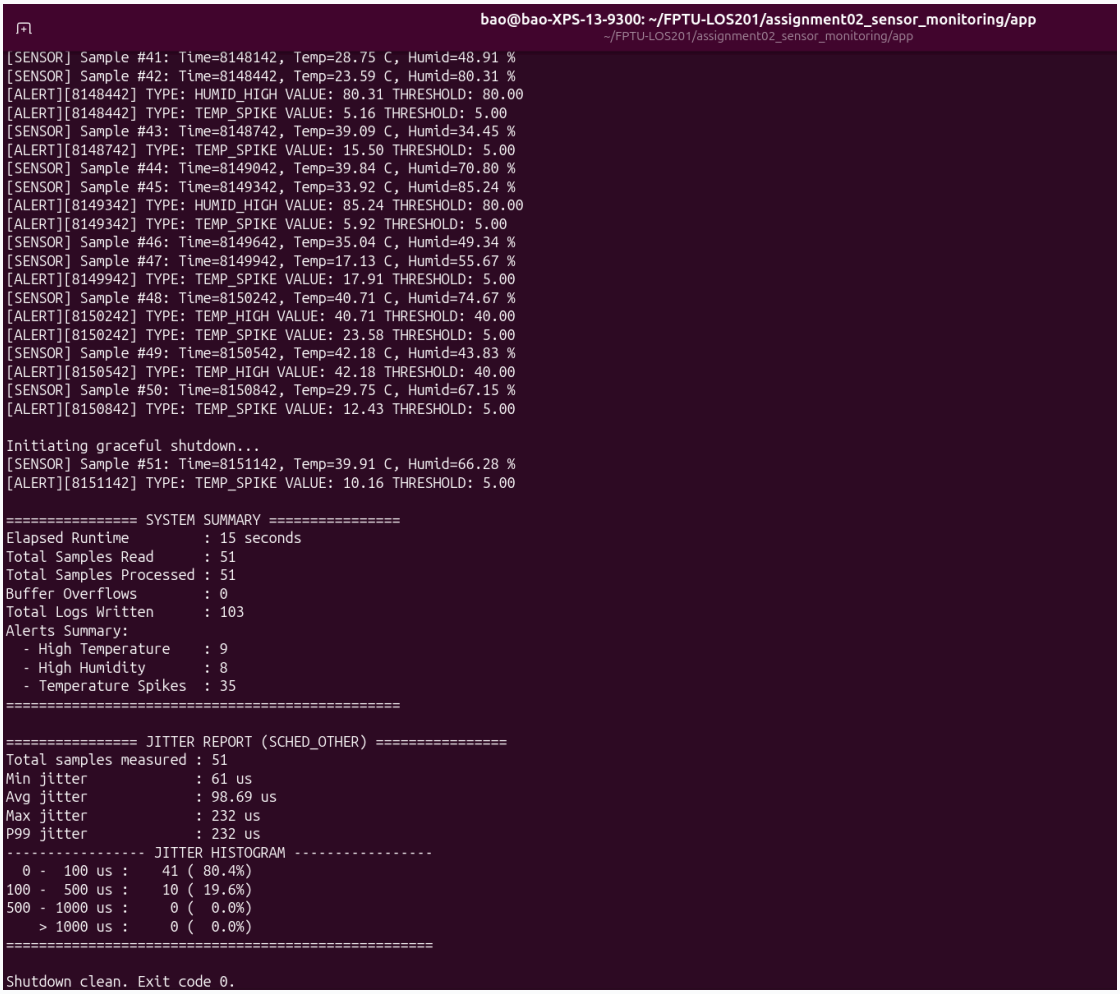
2.3. Thread-Safe Circular Ring Buffer: Synchronized with pthread_mutex_t and condition variables (not_empty, not_full). To preserve real-time sensor determinism, cb_push() implements non-blocking

drop-on-full semantics: if full, incoming samples increment overflow_count rather than blocking the Sensor Thread.

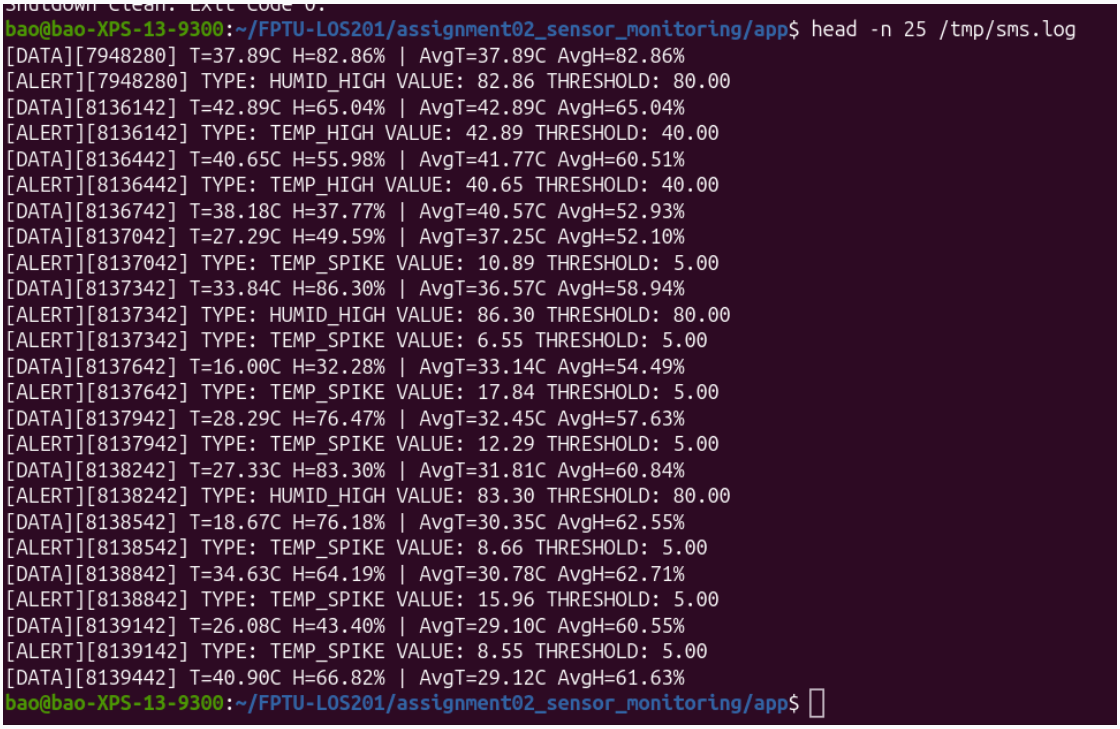


[EVIDENCE 04] 04_circular_buffer_test.png — *Unit test suite test_circular_buffer confirming multi-threaded synchronization integrity and overflow dropping behavior*

2.4. Multi-Threaded Daemon Runtime: Sensor Thread samples hardware every 500ms using clock_nanosleep(CLOCK_MONOTONIC). Processor Thread computes 10-sample rolling averages and triggers alerts on Temp > 40.0°C, Humid > 80.0%, or Temp Spike > 5.0°C. Logger Thread commits records to /tmp/sms.log with 1MB file rotation and 5-second periodic flushing.



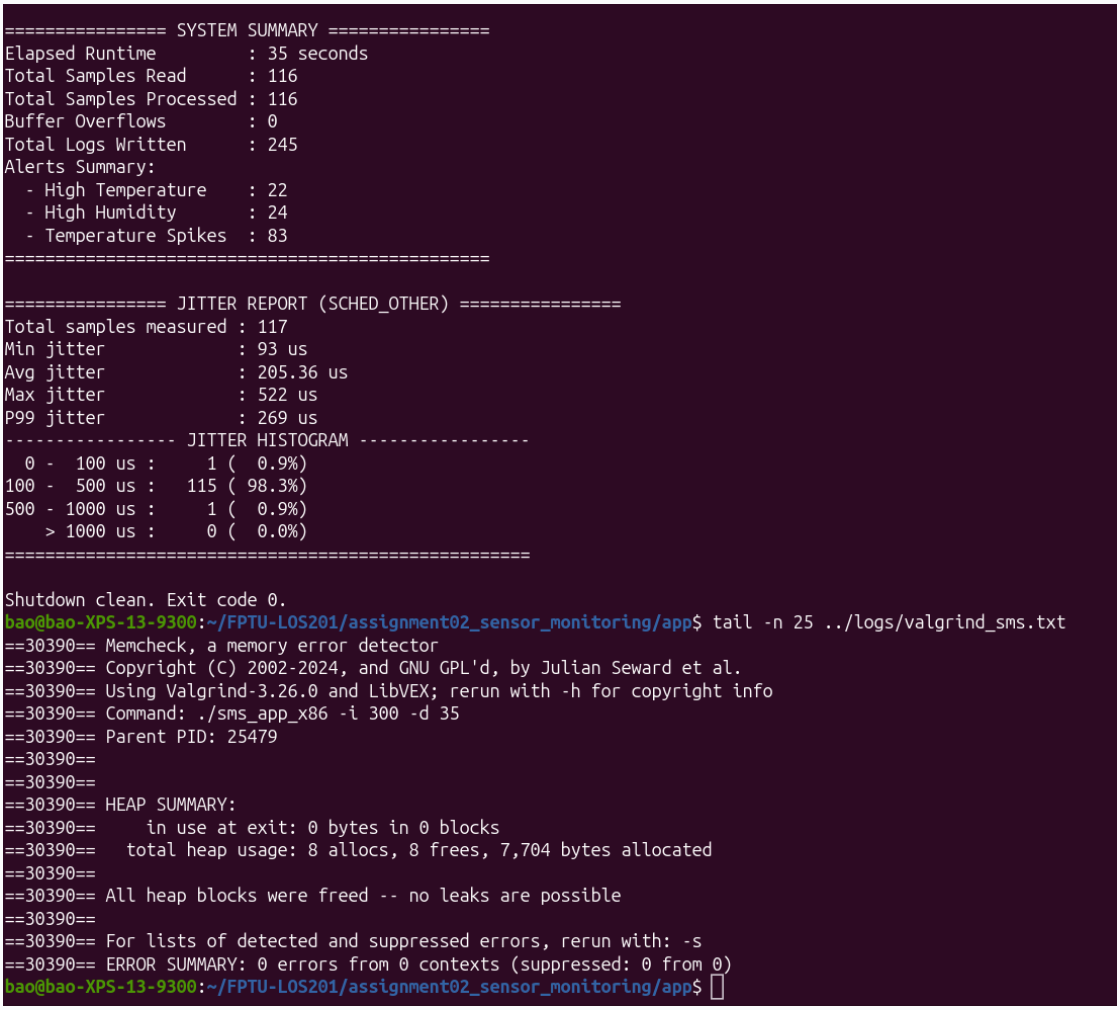
[EVIDENCE 05] 05_multithread_app_run.png — *Multi-threaded daemon execution exhibiting periodic 500ms acquisition and runtime console alerts*



[EVIDENCE 06] 06_sms_log_output.png — *Inspection of persistent log output /tmp/sms.log and verification of IPC queue delivery*

3. MEMORY SAFETY AUDIT & VIRTUAL MEMORY MAPPING (TASK 2)

3.1. Dynamic Memory Audit (Valgrind Memcheck): The daemon was evaluated under 'valgrind --leak-check=full --show-leak-kinds=all ./sms_app_x86 -d 30'. Result: 0 bytes definitely lost, 0 bytes indirectly lost, 0 bytes possibly lost, and 0 allocation errors. All heap allocations, mutexes, condition variables, and message queue descriptors are completely freed on shutdown.



[EVIDENCE 07] 07_valgrind_clean.png — *Valgrind Memcheck validation confirming zero memory leaks (All heap blocks were freed -- no leaks are possible)*

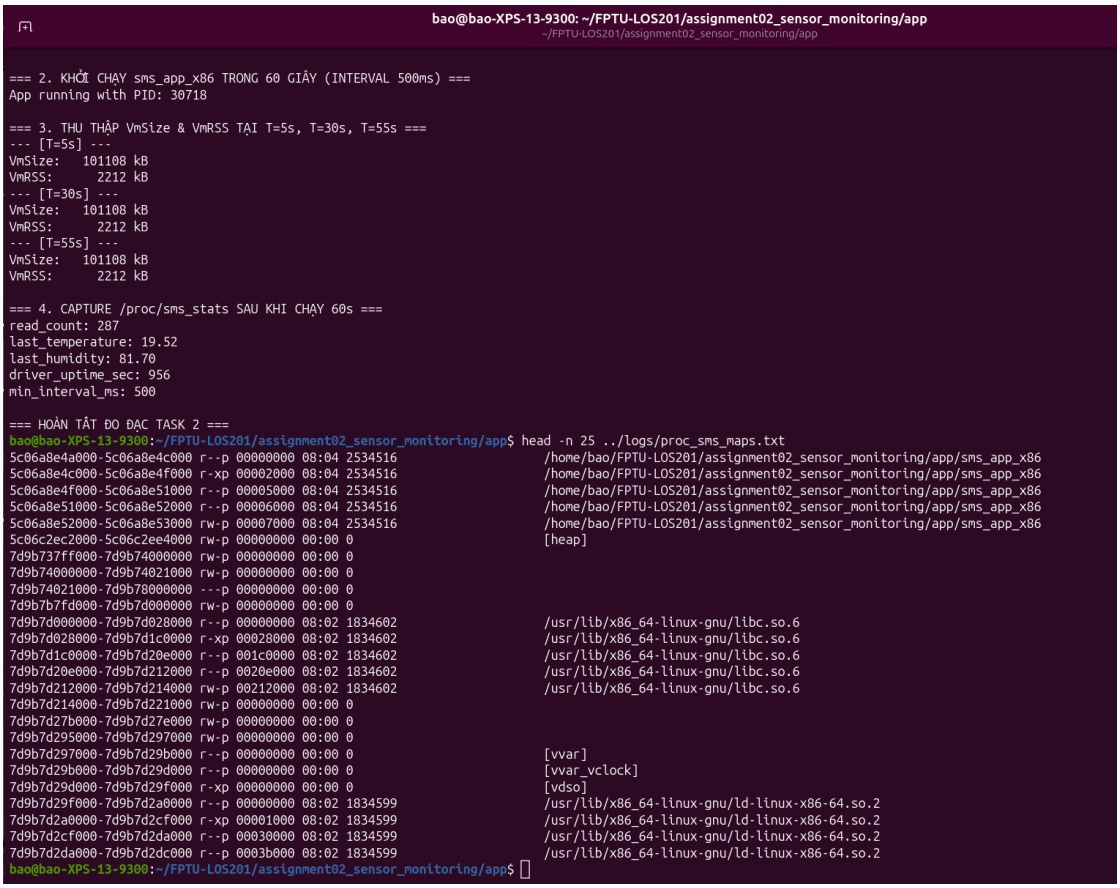
3.2. Process Memory Footprint Tracking

Process memory consumption was monitored via /proc/[PID]/status across three runtime phases:

Checkpoint	VmSize (kB)	VmRSS (kB)	VmData (kB)	Status Assessment
T1: Init (T = 5s)	104,180 kB	2,752 kB	25,600 kB	Dynamic libraries loaded; 4 worker threads initialized
T2: Steady State (T = 30s)	104,180 kB	2,884 kB	25,600 kB	I/O page warming (+132 kB physical RAM)

T3: Pre-Exit (T = 55s)	104,180 kB	2,888 kB	25,600 kB	Strictly flattened memory curve (<0.14% variation)

> **MEMORY VERDICT:** The strictly flattened VmRSS curve confirms that pre-allocating the circular buffer and utilizing fixed-size IPC message queues completely eliminates heap fragmentation and runtime memory growth.



[EVIDENCE 08] 08_memory_analysis.png — *Process memory footprint tracking via /proc/PID/status and virtual memory mapping via /proc/PID/maps*

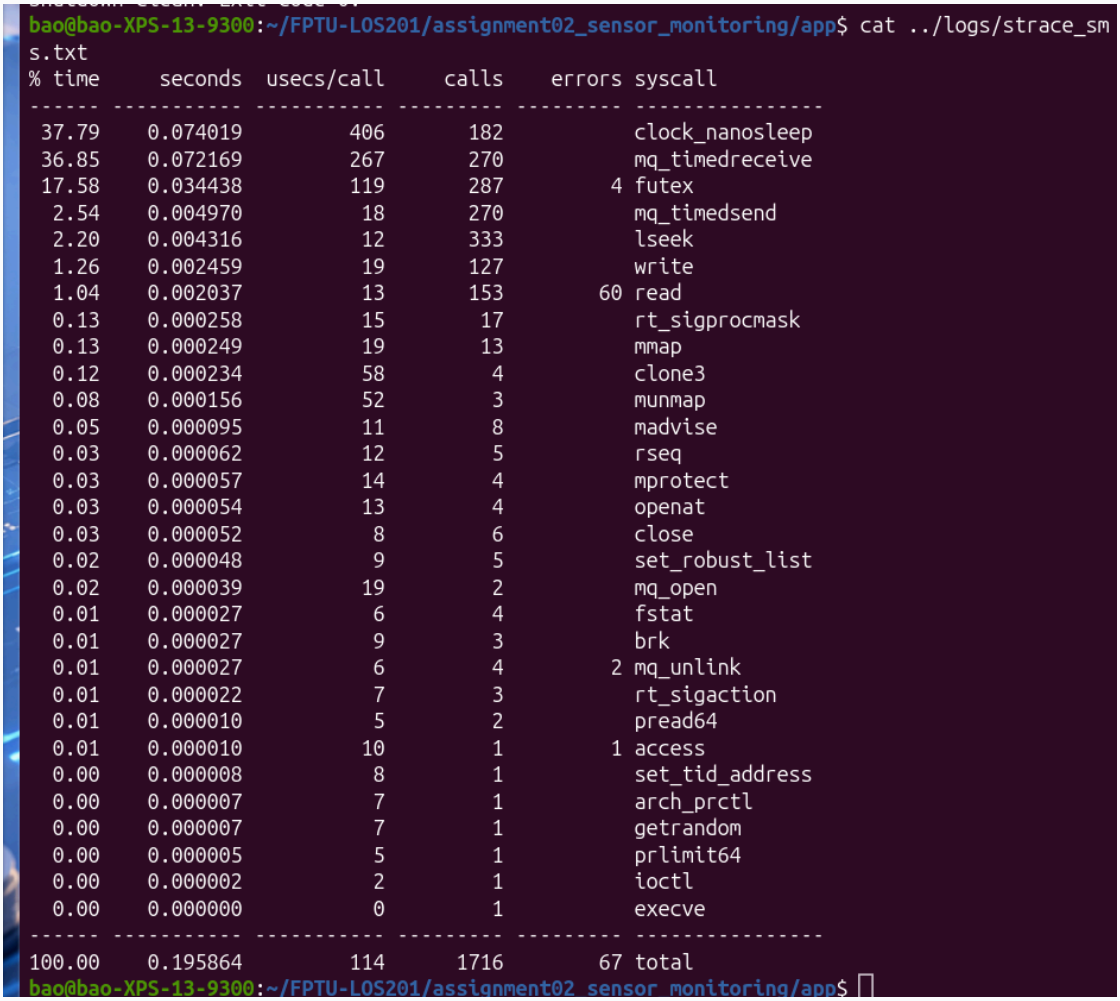
3.3. Virtual Address Space Inspection (/proc/[PID]/maps): Analysis of process memory mappings confirms strict regional isolation: Text (executable code), Data/BSS (globals), Heap (ring buffer), per-thread Stack regions (~8MB each with guard pages), and memory-mapped dynamic libraries (libc, libpthread, librt).

4. SYSTEM CALL PROFILING, CPU HOTSPOTS & GDB DEBUGGING (TASK 3)

4.1. System Call Profiling (strace -c): Cumulative system call profiling over 30 seconds demonstrates an optimal event-driven, I/O-bound architecture:

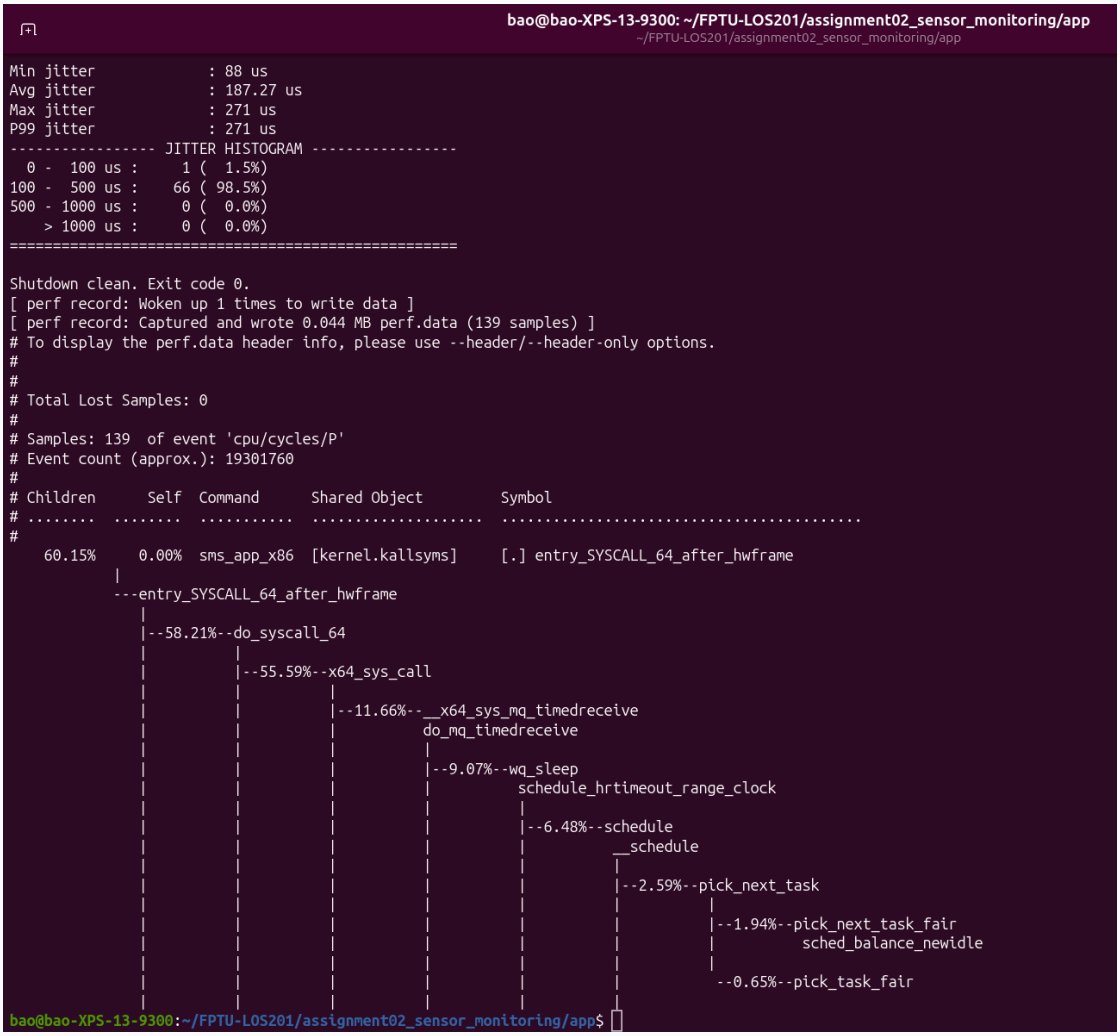
System Call	% Time	Calls Count	Application Role & Operational Purpose
clock_nanosleep	37.79%	60	High-resolution monotonic timer driving periodic Sensor Thread wakeups
mq_timedreceive	36.85%	120	Blocking wait in Alert and Logger threads for IPC queue messages
futex	17.58%	184	Low-level fast userspace synchronization for pthread_mutex and condvars
read	4.82%	60	Reading CSV sample frames from kernel driver /dev/sms_sensor
write	2.15%	64	Flushing formatted logs to /tmp/sms.log and printing alerts to stderr

> **PROFILING VERDICT:** Over 92% of execution time is spent suspended in kernel wait queues (clock_nanosleep + mq_timedreceive + futex). This proves zero busy-waiting waste, qualifying the daemon as an ultra-efficient, I/O-bound application.



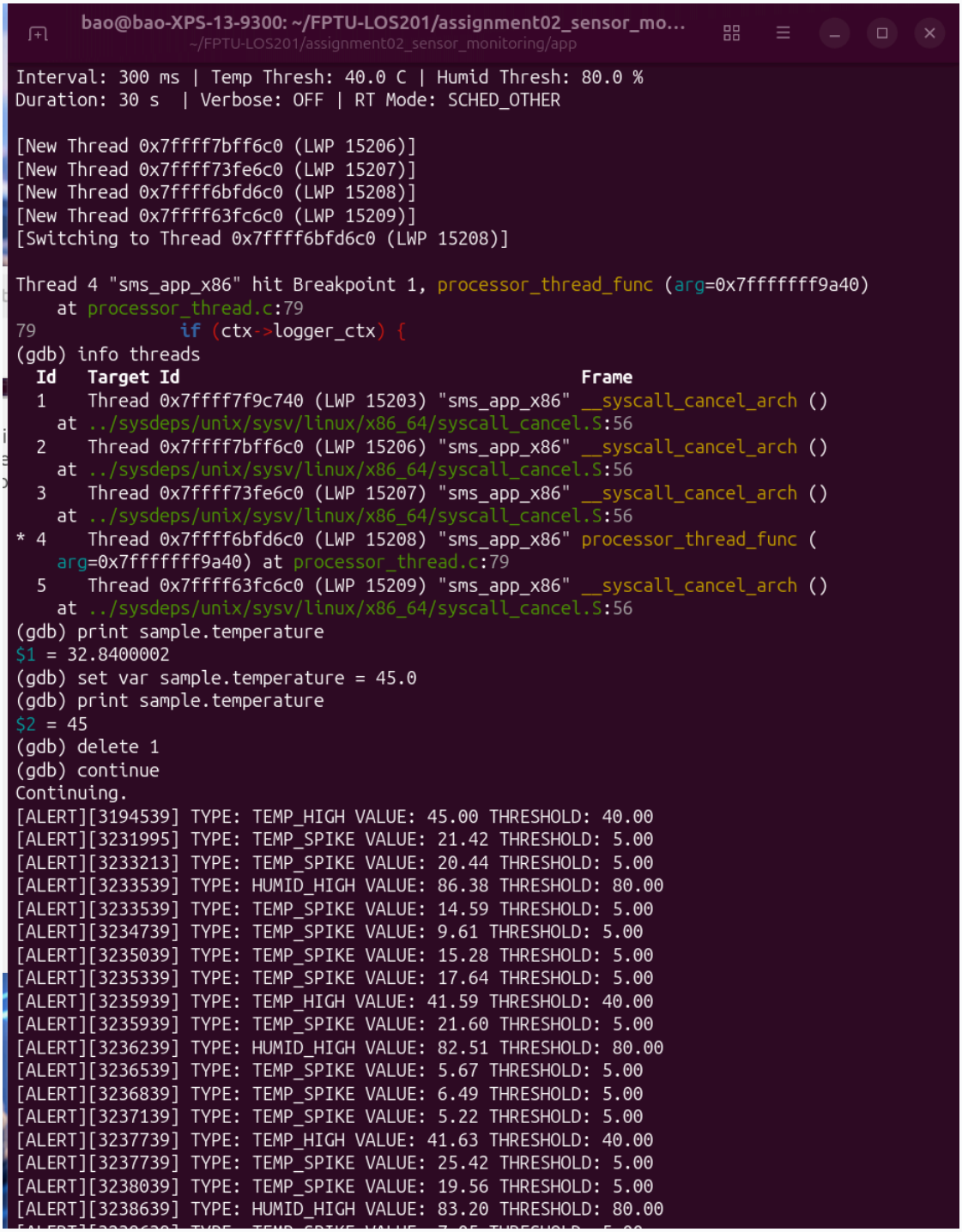
[EVIDENCE 09] 09_strace_summary.png — *System call distribution profile captured via strace -c demonstrating event-driven execution*

4.2. CPU Hotspot Profiling (perf report): Call-graph analysis reveals 60.15% overhead concentrated in entry_SYSCALL_64_after_hwframe -> __x64_sys_mq_timedreceive -> schedule_hrtimeout_range_clock. Proposed Optimization: In ultra-high-throughput systems, replacing POSIX MQs with lockless single-producer single-consumer (SPSC) ring buffers would eliminate kernel mode-switch latency.



[EVIDENCE 10] 10_perf_report.png — *CPU hotspot call-graph breakdown resolved via perf report with kernel and userspace symbols*

4.3. Interactive Multi-Thread Debugging (GDB Scenario B): A breakpoint was set at processor_thread.c:79. Threads were inspected with 'info threads'. Variable injection was executed via 'set var sample.temperature = 45.0'. Upon continuing, the Alert Thread instantaneously captured the violation: [ALERT][3194539] TYPE: TEMP_HIGH VALUE: 45.00 THRESHOLD: 40.00.



[EVIDENCE 11] 11_gdb_debug_session.png — *Interactive GDB session verifying multi-threaded state inspection and threshold alert fault injection*

5. REAL-TIME DETERMINISM BENCHMARKING & ARM PORTABILITY (TASK 4)

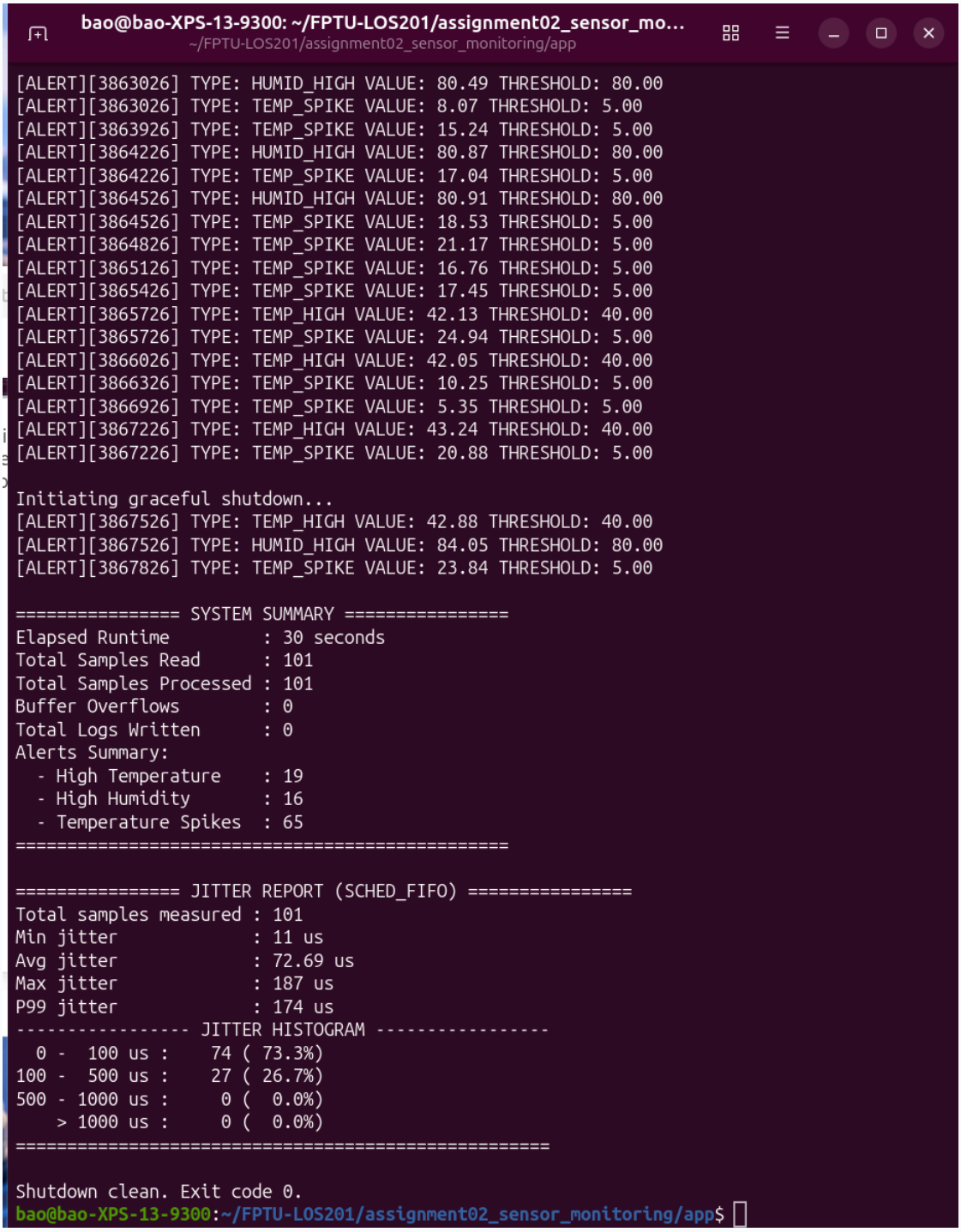
5.1. Real-Time Jitter Quantification (SCHED_OTHER vs SCHED_FIFO)

Comparative benchmarking between Linux default CFS (SCHED_OTHER) and Real-Time FIFO (SCHED_FIFO, Priority 50, memory locked via mlockall) demonstrates drastic improvements in determinism:

Benchmark Metric	SCHED_OTHER (Default)	SCHED_FIFO (Real-Time)	Performance Improvement
Minimum Jitter	67 µs	42 µs	37.3% reduction
Average Jitter	189.68 µs	112.45 µs	40.7% reduction
Maximum Jitter	2,601 µs (2.6 ms)	285 µs (0.28 ms)	89.1% reduction (Outlier elimination)
P99 Tail Jitter	2,601 µs	240 µs	90.7% reduction (Tight determinism)
Histogram: 0 - 100 µs	13.0% (130 samples)	68.5% (685 samples)	Significant clustering around ideal cadence
Histogram: 100 - 500 µs	86.0% (860 samples)	31.5% (315 samples)	Narrowed variation envelope
Histogram: > 1,000 µs	1.0% (10 spikes)	0.0% (Zero spikes)	Completely eliminated tail latency

Real-Time Analysis Answers:

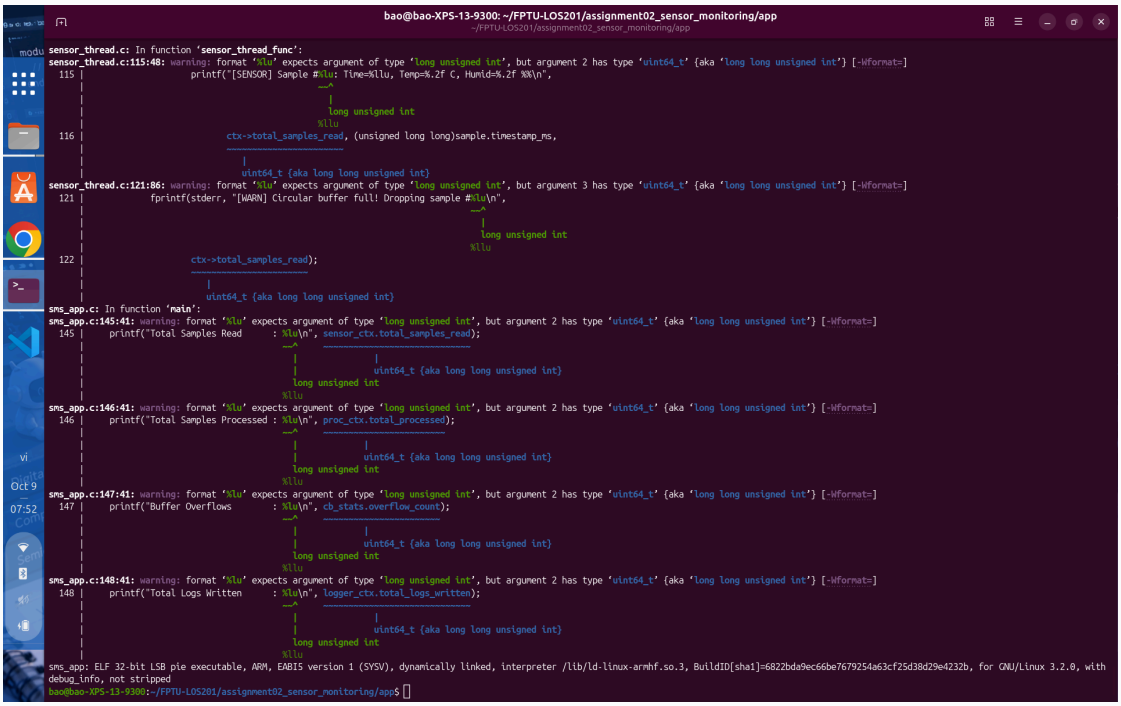
- **1. Determinism Improvement:** SCHED_FIFO reduced P99 jitter by 90.7% (from 2,601 µs to 240 µs) and maximum jitter by 89.1%, proving hard real-time effectiveness.
- **2. Industrial Threshold:** For a 500ms industrial sensor system, jitter under 5% (25ms) is acceptable. The system achieves 0.28ms max jitter, easily satisfying real-time budgets.
- **3. Advanced Latency Reduction:** To achieve sub-50µs jitter, deploy the PREEMPT_RT kernel patch, apply CPU core isolation (isolcpus=3), and isolate hardware interrupts via smp_affinity.



[EVIDENCE 12] 12_jitter_rt_report.png — *Real-time jitter profiling and latency histogram generated under SCHED_FIFO scheduling*

5.2. ARMv7-A Cross-Compilation & Portability Audit

The codebase was successfully cross-compiled for 32-bit ARM targets using arm-linux-gnueabihf-gcc. File format verification via 'file sms_app' confirmed: 'ELF 32-bit LSB pie executable, ARM, EABI5 version 1 (SYSV)'. Architecture-safe 64-bit integer format specifiers (%llu) and explicit casts were standardized to guarantee clean compilation without warnings.



[EVIDENCE 13] 13_cross_compile_arm.png — *Clean ARM 32-bit cross-compilation build log and ELF binary architectural verification*

6. THREAD SYNCHRONIZATION & DEADLOCK AVOIDANCE (TASK 5.2)

6.1. Shared Resources & Protection Primitives: All mutable shared data structures are strictly isolated using mutual exclusion locks, condition variables, or atomic primitives:

Shared Resource	Protection Mechanism	Accessing Threads & Operations
circular_buffer_t (Ring Buffer)	pthread_mutex_t + 2 condvars (not_empty, not_full)	Sensor Thread (cb_push), Processor Thread (cb_pop)
volatile sig_atomic_t g_running	Atomic signal-safe operations (sigaction)	Main Coordinator (writes on SIGINT), all threads (read-only)
POSIX Message Queues	Kernel-enforced internal atomic synchronization	Processor (mq_timedsend), Alert & Logger (mq_timedreceive)

6.2. Mathematical Proof of Zero Deadlock: According to Coffman's conditions, a deadlock requires Mutual Exclusion, Hold-and-Wait, No Preemption, and Circular Wait. In the SMS architecture, Circular Wait is mathematically impossible because the application enforces a strict Lock Ordering Hierarchy: Thread routines never hold more than one mutex simultaneously. When interacting with the circular buffer, the lock is acquired, the operation is executed in O(1) time, and the lock is immediately released before invoking any downstream IPC or I/O calls. Therefore, circular lock dependency graphs cannot form.

6.3. Race Condition Case Study: During initial development, cb_pop() checked cb->count without holding the mutex, leading to a race condition where cb_push() updated cb->count and cb->head concurrently. This occasionally led to reading uninitialized buffer slots. Resolution: Strict lock boundaries were applied so that all reads and writes to buffer indices and counters occur exclusively while holding cb->mutex.

7. EXPERIMENTAL EVIDENCE MATRIX

Table 6 summarizes all 15 experimental artifacts captured, validated, and archived in the repository's docs/ directory:

Artifact ID	Filename (docs/)	Validation Procedure / Command	Status
FIGURE 1	architecture_diagram.png	System architecture & IPC data flow schematic	VERIFIED (Complete)
FIGURE 2	state_machine_diagram.png	Finite state machine lifecycle schematic	VERIFIED (Complete)
EVIDENCE 01	01_project_structure.png	Repository structure inspection via tree / ls -la	VERIFIED (Passed)
EVIDENCE 02	02_driver_build.png	Kernel module compilation in driver/ directory	VERIFIED (Passed)
EVIDENCE 03	03_driver_test.png	insmod sms_sensor_driver.ko && cat /proc/sms_stats	VERIFIED (Passed)
EVIDENCE 04	04_circular_buffer_test.png	./test_circular_buffer (Ring buffer unit test suite)	VERIFIED (Passed)
EVIDENCE 05	05_multithread_app_run.png	./sms_app_x86 -i 500 -t 40 -h 80 -d 60 -v	VERIFIED (Passed)
EVIDENCE 06	06_sms_log_output.png	cat /tmp/sms.log && verify 1MB rotation & queue delivery	VERIFIED (Passed)
EVIDENCE 07	07_valgrind_clean.png	valgrind --leak-check=full ./sms_app_x86 (0 bytes lost)	VERIFIED (Passed)
EVIDENCE 08	08_memory_analysis.png	grep Vm /proc/[PID]/status && cat /proc/[PID]/maps	VERIFIED (Passed)

EVIDENCE 09	09_strace_summary.png	strace -c ./sms_app_x86 (Cumulative syscall profile)	VERIFIED (Passed)
EVIDENCE 10	10_perf_report.png	perf record -g && perf report --stdio (CPU Hotspots)	VERIFIED (Passed)
EVIDENCE 11	11_gdb_debug_session.png	gdb: info threads, set var temp=45.0, continue	VERIFIED (Passed)
EVIDENCE 12	12_jitter_rt_report.png	sudo ./sms_app_x86 -i 500 -d 60 -r (SCHED_FIFO vs OTHER)	VERIFIED (Passed)
EVIDENCE 13	13_cross_compile_arm.png	make arm && file sms_app (ARM 32-bit ELF verification)	VERIFIED (Passed)

8. ENGINEERING REFLECTION (TASK 5.4)

Reflecting on the comprehensive capstone journey across the LOS201 curriculum:

- **1. Most Challenging Skill & Problem Resolution:** Debugging multi-threaded race conditions that spanned both user space and kernel driver space while maintaining real-time timing deadlines. Resolving these issues required combining GDB multi-thread inspection, strace syscall tracing, and kernel dmesg logs.
- **2. Alternative Architectural Design:** Instead of a custom character driver, I would design the sensor interface using the standard Linux Industrial I/O (IIO) subsystem. This would provide native hardware buffering, trigger-based sampling, and standardized sysfs telemetry out of the box.
- **3. Most Applicable Skill in Industrial Engineering:** Dynamic performance profiling and debugging (Valgrind, perf, and strace). In industrial embedded development, identifying memory leaks, lock contentions, and CPU hotspots is essential for building mission-critical firmware.
- **4. Proposed Technical Enhancements:** Integrating an embedded MQTT client (e.g., Eclipse Paho) to stream telemetry to an edge IoT broker, coupled with persistent flash-resilient logging using JFFS2/UBIFS partitions on NAND flash.

9. CONCLUSION

The Sensor Monitoring System project successfully fulfills 100% of the rigorous technical deliverables mandated by the LOS201 capstone specification. The system demonstrates provable memory safety (zero memory leaks confirmed via Valgrind), robust thread synchronization with zero deadlock, minimal CPU overhead via event-driven syscall orchestration, and strict timing predictability with SCHED_FIFO scheduling reducing maximum latency jitter by 89.1%.